

LANCE: An LLM Agent for Network Compromise Evaluation with IoTChainBench

Anonymous Authors

Paper #XXXX, Anonymized for Double-Blind Review

Abstract—LLM penetration-testing agents are typically evaluated on isolated hosts, while IoT compromises often depend on reachability across gateways, brokers, and cameras. This leaves a methodological gap: single-host corpora do not measure pivots, protocol roles, or evidence depth.

We present IoTChainBench, a reproducible network-scale IoT benchmark of 12 Proxmox/Ansible scenarios with per-vulnerability ground truth and structured metadata for evaluating findings across devices and network paths. We also introduce LANCE (LLM Agent for Network Compromise Evaluation), a six-phase network-native harness combining topology analysis, active discovery, triage, verification, intrusion attempts, and report generation.

In a controlled comparison under the same model and evaluator, LANCE achieves 0.935 F1 in informed mode (topology prior) and 0.887 F1 in blind mode (active discovery only), outperforming both adapted single-host baselines on all 12 scenarios. On external single-host benchmarks, LANCE reaches 27.3% end-to-end success on AutoPenBench and 30.2% useful findings on a 328-case Vulnhub snapshot.

Index Terms—IoT security, penetration testing, large language models, autonomous agents, benchmark, cyber-physical systems

1. Introduction

IoT deployments are compromised through paths, not just devices. Mirai [1], Verkada [2], and Stuxnet [3] each demonstrate this: impact came from chained access across cameras, brokers, PLCs, and segmented boundaries.

Yet LLM penetration-testing agents are evaluated on a single host at a time: isolated containers, VMs, or CTF services scored independently. This gap is methodological, not merely architectural. Single-host corpora cannot measure three capabilities central to IoT/OT security: finding vulnerabilities behind pivots, reasoning over domain protocols (MQTT, Modbus, CoAP, LoRaWAN), and requiring proof before counting a finding. Without a network-scale benchmark, strong single-host scores overstate readiness for real deployments.

Contributions. This paper makes two main contributions.

- **IoTChainBench:** a reproducible network-scale benchmark that tests multi-hop IoT/OT compromise across gateways, brokers, cameras, sensors, and service dependencies. Each scenario is deployed via Ansible and

includes per-vulnerability ground truth for evaluating agent findings.

- **LANCE:** the LLM Agent for Network Compromise Evaluation, a six-phase network-native harness (topology → discovery → triage → verification → intrusion → report) for multi-hop IoT penetration testing. LANCE requires evidence before counting a vulnerability as found and achieves 0.935 F1 on the 12 evaluated scenarios, while recovering 86.4% of the vulnerability severity mass.

All artifacts are released at ANONYMIZED_URL.

2. Related Work

This section reviews prior work relevant to our approach. We first survey LLM-driven penetration-testing agents and the benchmarks used to evaluate them, then discuss the classical attack-graph literature that informs LANCE’s topology-aware design, and finally review IoT firmware analysis and the security frameworks underlying our scenario taxonomy.

2.1. LLM Penetration-Testing Agents

Single-agent systems. Happe and Cito [4] were among the first to show that a single LLM driving a shell can autonomously perform privilege escalation on Linux VMs. PENTESTGPT [5] improved this with a three-module design over a *Pentesting Task Tree*, outperforming direct GPT-3.5 and GPT-4 usage on HackTheBox and VulnHub targets. HACKINGBUDDYGPT [6], AUTOATTACKER [7], and NEBULA [8] extend this line as open-source tools.

Multi-agent pipelines. VULNBOT [9] decomposes pentesting into reconnaissance, scanning, and exploitation over a *Penetration Task Graph*, gaining 21 points end-to-end over its single-agent baseline (30.3% on AutoPenBench with Llama 3.1-405B). AUTOPENTESTER [10] adds RAG across five agents for a +27% subtask gain over PENTESTGPT on HackTheBox. PENTESTAGENT [11] and CAI [12] generalise this blueprint with modular tool use and a public leaderboard.

Model-level improvements. PENTEST-R1 [13] applies two-stage reinforcement learning on reasoning traces. XOFFENSE [14] fine-tunes Qwen3-32B on Chain-of-Thought penetration testing data, reaching 72.72% task completion

and 79.17% best-of-five subtask completion on AutoPenBench. CHECKMATE [15] couples an LLM with a *Classical Planning+* module, reaching an interactive shell (milestone M7) on 88% of 120 Vulnhub targets.

Common limitation. Despite this diversity, reported evaluations remain single-target or challenge-level. They do not score subnet reachability, pivot depth, or per-vulnerability progress across IoT/OT protocols (MQTT, Modbus, LoRaWAN, CoAP), which is the gap IOTCHAINBENCH and LANCE address.

2.2. LLM Cybersecurity Benchmarks

Challenge-level corpora. AUTOPENBENCH [16] is the dominant evaluation substrate: 33 Docker-container tasks with milestone and flag-based scoring, used by VULNBOT, PENTEST-R1, and XOFFENSE. VULHUB [17] provides reproducible vulnerable environments used by CHECKMATE. CAIBENCH [18], NYU CTF BENCH [19], and CYBENCH [20] extend coverage to CTFs, attack-and-defense ranges, and knowledge tests. These corpora score at challenge or flag level. They do not provide per-vulnerability ground truth tied to reachability, hop depth, or IoT/OT protocols.

Multi-host labs. LUDUS [21] and GOAD [22] deliver Proxmox+Ansible deployments of enterprise Active Directory networks. They are labs and training ranges, not scored LLM benchmarks. Both model multi-host compromise and lateral movement, but neither ships a per-vulnerability ground-truth schema, hop-depth annotations, or an evaluator. Both also target enterprise AD rather than IoT/OT protocols. To our knowledge, no prior benchmark combines multi-host reachability, IoT/OT protocol coverage, per-vulnerability ground truth, and evidence-level scoring beyond binary flag capture (Table 1).

2.3. Attack Graphs and Network Reachability

Representing a network as a graph for security analysis predates deep learning by decades. Phillips and Swiler [23] introduced graph-based network-vulnerability analysis. Sheyner *et al.* [24] developed automated attack-graph generation via model checking, and MULVAL [25] cast the problem as Datalog inference. These systems compute reachable attacker states from a *static* encoding of host configurations and exposed services. LANCE borrows this reachability abstraction but operates on a *live* network: edges are validated by probes, the LLM proposes the next action, and the graph is updated as hosts are discovered. Classical attack-graph generation is therefore complementary rather than competing.

2.4. IoT Security Context

Scenario design and ground-truth taxonomy follow three references: OWASP IoT Top 10 [26] for vulnerability cate-

gories, MITRE ATT&CK for ICS [27] for OT-tactic annotations, and NIST SP 800-82 Rev.3 [28] for OT security and segmentation conventions.

Empirical IoT-security artefacts fall into two families. **Network-traffic datasets** (IoT-23 [29], TON_IoT [30], Edge-IIoTset [31]) provide labeled traces for intrusion detection. They are not deployable pentest targets with per-vulnerability ground truth or controllable reachability. **Firmware-focused work** (Costin *et al.* [32], FIRMA-DYNE [33], FIRMAE [34], IOTGOAT [35]) exercises single-device firmware analysis. These datasets measure detection or firmware-analysis capability. IOTCHAINBENCH instead measures active security assessment on deployed multi-device networks with firewall-enforced reachability.

TABLE 1. GAP ANALYSIS ACROSS LLM PENTEST AGENTS AND CORPORA. GT = GROUND-TRUTH ORACLE TYPE (FLAG / TASK-STEP / PER-VULN / NONE); MH = MULTI-HOP NETWORK REASONING; IoT/OT = COVERS IoT OR OT PROTOCOLS.

System	Scale	IoT/OT	GT	MH
<i>LLM pentest agents</i>				
Happe & Cito [4]	1 host	✗	flag	✗
PentestGPT [5]	1 host	✗	task-step	✗
VulnBot [9]	1 host	✗	task-step	✗
AutoPentester [10]	1 host	✗	task-step	✗
CAI [12]	1 host	✗	flag	✗
Pentest-R1 [13]	1 host	✗	task-step	✗
xOffense [14]	1 host	✗	task-step	✗
EniGMA [36]	1 host	✗	flag	✗
CheckMate [15]	1 host	✗	task-step	✗
<i>Evaluation corpora</i>				
AutoPenBench [16]	1 host	✗	flag	✗
CAIBench / RCTF2 [18]	1 host	partial	flag	✗
Vulnhub [17]	1 host	✗	none	✗
Ludus + GOAD [21], [22]	network	✗	none	✗
IOTChainBench (ours)	network	✓	per-vuln	✓

3. Threat Model and Problem Definition

Problem statement. Given a target subnet with a designated entry point e declared in each scenario’s topology metadata, *network-scale penetration testing* is the task of discovering all hosts reachable from e , identifying their vulnerabilities, producing evidence of exploitability, and reporting findings across the full network. The unit of evaluation is the network, not an individual host: IoT risk is driven by pivot chains (e.g., entry point $\rightarrow$ gateway $\rightarrow$ MQTT broker $\rightarrow$ OT controller), and single-host metrics are blind to vulnerabilities that only become reachable after one or more pivots.

Attacker model. The attacker is an external adversary with Layer-3 access to the target subnet but no prior credentials, no out-of-band knowledge of deployed software, and no privileged vantage beyond active scanning. Their objective is data exfiltration (sensor readings, credentials) or OT disruption (reaching industrial-control services behind a segmentation boundary). They may use standard pentest tooling (Nmap, MQTT clients, SSH, HTTP) augmented by an LLM with tool-calling, within a bounded turn budget

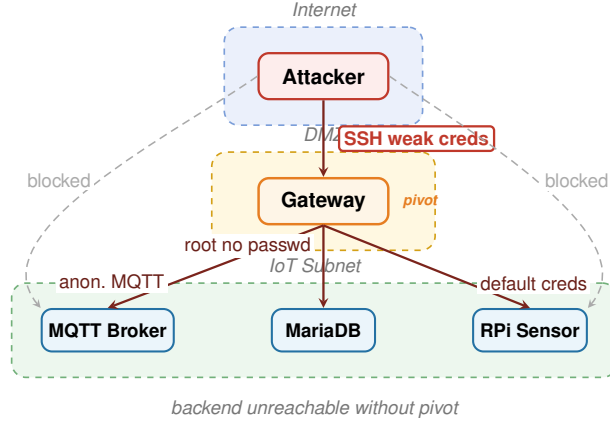


Figure 1. Gateway-centric attack chain (Scenario S3). Backend services are unreachable from the Internet without first pivoting through the gateway. Single-host agents fail on this topology (F1=0 for both baselines) because they scan each target independently.

(§5.2). This adversarial setting requires multi-step reasoning across heterogeneous protocols, which motivates the use of an LLM-driven agent rather than a static scanner.

Evidence levels. A finding is solved at the deepest level the agent can demonstrate within its turn budget:

- **L1 detection:** a port scan or banner confirms the issue (e.g., unauthenticated MQTT port 1883 open).
- **L2 exploitation:** an active interaction demonstrates it (e.g., anonymous MQTT subscribe on #, SSH login with default credentials).
- **L3 exfiltration:** sensitive content is retrieved (e.g., MariaDB root dump, camera RTSP stream).

This ordering governs both LANCE Phase 4 prompting (§5.1) and evaluator scoring (§4.5).

Scope. We evaluate at the IP/application layer on virtual IoT networks. Out of scope: physical and side-channel attacks, RF-layer exploitation, firmware extraction, post-exploitation persistence, and insider threats.

Figure 1 illustrates a representative gateway-centric scenario: the attacker can reach the gateway directly but all backend services are unreachable without first compromising it.

4. IoTChainBench: A Network-Scale Benchmark

4.1. Design Principles

IoTCHAINBENCH is built around six principles:

- **Reproducibility.** Full infrastructure deploys from Ansible playbooks against a Proxmox cluster; vulnerabilities are injected idempotently with no manual steps.
- **Architectural diversity.** Scenarios span five network patterns with reachability enforced by OpenWrt firewall rules in segmented/VLAN cases.

- **Protocol coverage.** Each scenario mixes IoT and OT protocols (MQTT, Modbus, LoRaWAN, CoAP, SNMP, SSH, HTTP, FTP, Telnet) to exercise protocol-specific reasoning.
- **Ground-truth granularity.** Every vulnerability is documented in YAML with device, IP, severity, OWASP IoT and MITRE ICS mappings, optional CVE, indicators, and a verification command.
- **Scalability axis.** Ten main scenarios (4–8 devices) and two scalability scenarios (15 and 35 devices) separate pattern coverage from network size.

4.2. Network Topologies and Scenario Catalogue

IoTChainBench defines five topological patterns that vary the number of pivots required to reach the deepest vulnerable service, directly setting the pivot-depth difficulty axis:

- 1) **P1 — Flat.** Single subnet; all devices directly reachable from the entry point.
- 2) **P2 — Gateway-centric.** One IoT gateway mediates all back-end access; back-end services unreachable without gateway compromise.
- 3) **P3 — Segmented IT/OT.** Admin/IT/OT zones with ACLs; the OT zone (Modbus PLCs, HMI) requires a jump-host pivot.
- 4) **P4 — Hub-centric star.** A central hub is the sole path to the MQTT broker, DB, and camera; peripherals are mutually isolated.
- 5) **P5 — Edge-cloud pivot.** Edge and cloud subnets are separated; only the edge gateway bridges them.

The 12 scenarios (Table 2) are organised along two axes: 10 *main* scenarios instantiate the patterns above at 4–8 devices, and 2 *scalability* stressors push to 15 and 35 devices using larger multi-zone variants.

TABLE 2. IOTCHAINBENCH SCENARIO CATALOGUE.

ID	Pattern / role	Dev	Vulns	Theme
<i>Main scenarios (pattern coverage)</i>				
s_1	P1 Flat	4	12	Minimal IoT (MQTT/Web/SSH)
s_2	P1 Flat	6	13	Flat IoT + IT mix
s_3	P2 Gateway-centric	8	18	NATO lab replica
s_4	P3 Segmented IT/OT	8	18	Industrial (Modbus/HMI)
s_5	P4 Hub-centric star	8	15	Smart building (cameras/NVR)
s_6	P4 Hub-centric star	6	16	Home automation hub
s_7	P5 Edge-cloud pivot	6	14	Edge → cloud API
s_8	P3 Segmented IT/OT	8	14	IT/IoT/OT 3-zone industrial
s_9	P1 Flat (mesh)	6	11	Mesh IoT smart-city outdoor
s_10	P1 Flat (variants)	6	13	Role-variant stress
<i>Scalability stressors</i>				
s_11	Smart City (flat)	15	23	3 zones same /24 (no isolation)
s_12	Smart City (large)	35	42	Largest network (34 LXC services)
Total		116	209	

4.3. Deterministic Vulnerability Injection

All vulnerabilities are injected by a dedicated Ansible role parameterized by `scenario_id`, ensuring fully reproducible and idempotent deployments. Each scenario YAML

declares router-level weaknesses (Telnet, WAN admin, FTP) and per-service role definitions that trigger targeted injections, covering the most common real-world IoT misconfigurations: anonymous MQTT access, weak SSH credentials, and passwordless database roots.

The 209 injected vulnerabilities span 11 categories (Table 3), dominated by misconfigurations (62), data exposure (36), and missing authentication (34). Severity skews high: 95 HIGH and 33 CRITICAL instances, reflecting realistic IoT deployment weaknesses rather than theoretical edge cases.

TABLE 3. VULNERABILITY CATEGORY AND SEVERITY DISTRIBUTION.

Category	Count	CRIT	HIGH	MED	LOW
misconfiguration	62	7	30	25	0
no_authentication	34	14	18	2	0
data_exposure	36	0	14	22	0
default_credentials	26	3	23	0	0
info_disclosure	13	0	0	0	13
code_injection	10	9	1	0	0
cve	9	0	3	6	0
insecure_update	9	0	4	5	0
weak_crypto	5	0	0	0	5
missing_header	3	0	0	0	3
privilege_escalation	2	0	2	0	0
Total	209	33	95	60	21

4.4. Ground-Truth Schema

Each vulnerability-injected scenario ships a YAML file with one entry per injected vulnerability. Each entry records the affected device, IP, role, severity, category, OWASP IoT Top 10 and MITRE ATT&CK for ICS mappings, an optional CVE, expected detection indicators, a verification command that confirms exploitability, and reachability metadata for pivot-aware analysis. This per-vulnerability granularity, absent from flag-capture benchmarks, supports the common matcher and severity-weighted score used in our evaluation.

```
- id: V1
  title: "MQTT without authentication"
  severity: high
  category: misconfiguration
  owasp_iot: "I1 / I9"
  mitre_ics: "Initial Access, Collection"
  indicators:
    - "allow_anonymous true"
  verification: "mosquitto_sub -h 192.168.100.11 -t '#' -v"
```

Figure 2. Example ground-truth entry.

Scenarios may additionally declare a `bonus_types` list of vulnerability types adjacent to the injected set. Agent findings that match these types are counted as neither True Positives nor False Positives, avoiding penalties for legitimate hardening recommendations while keeping the recall denominator fixed.

4.5. Scoring and Diagnostic Fields

Finding matcher. Each LLM finding is matched against ground truth in three ordered passes: (1) exact CVE match; (2) (ip, vuln_type) with canonicalised type; (3) (ip, severity) fallback. Unmatched findings are False Positives. Duplicate findings on (ip, type, port) are resolved by retaining the lower severity to counteract inflation observed in pilot runs.

Weighted score. The composite score is a severity-weighted True-Positive sum (w : critical 4, high 3, medium 2, low 1) normalized by $\sum_{g \in \text{GT}} w(g)$, with $0.5\times$ and $0.75\times$ penalties for fallback and severity-mismatch matches respectively. F1 treats each detected vulnerability as binary: found or not, regardless of the severity label the agent assigns. The CVSS-weighted score is more demanding: it weights each true positive by its ground-truth severity and applies a $0.75\times$ penalty when the agent’s predicted severity differs from the injected ground truth. Because LANCE infers severity from scan evidence and banner analysis rather than computing it from a CVSS vector, some misclassification is expected even when all vulnerabilities are detected.

Hop-depth metadata. For each ground-truth vulnerability g , the benchmark can store the minimum number of pivots from entry point e :

$$\text{hop_depth}(g) = \min_{p \in \text{paths}(e \rightarrow d_g)} \max(0, |p|_E - 1).$$

We use this metadata to design multi-hop scenarios and to interpret qualitative failure modes. We do not report a separate hop-depth aggregate in the present evaluation.

5. LANCE: An LLM Agent for Network Compromise Evaluation

Existing LLM pentest harnesses treat the network as a flat list of targets, letting a single agent accumulate unbounded context as the network grows. LANCE instead decomposes network pentesting into six phases with typed deliverables. The key design decision is *parallelization at the device and vulnerability granularity*: rather than a single monolithic agent holding the full network state, we spawn dedicated micro-agents per target with constrained tool sets and aggregate deterministically. This keeps each micro-agent’s context bounded at $O(1)$ in the number of network devices, with cross-device state re-introduced only in Phase 5 (intrusion) and Phase 6 (report). Vulnerability type strings produced by LLM agents are resolved through a canonical taxonomy of 11 categories (Table 3), filtering noise and deduplicating collisions before evaluation.

5.1. Pipeline

Phase 1: Topology prior. Phase 1 loads a directed graph $G = (V, E)$ of the target network, where V are devices and E are protocol-labeled links. In *informed* mode, the

topology is loaded from the benchmark YAML, giving the harness a full network prior before any scan; in *blind* mode, the graph starts empty and is populated by Phase 2 active discovery. The output enumerates the attack surface and ranks pivot candidates by betweenness centrality (identifying devices whose compromise enables the most lateral paths).

Phase 2: Network discovery. Phase 2 executes active reconnaissance across the target /24 using host discovery, service/version scanning, L2 vendor probing, and protocol-specific probes (MQTT, SSH, HTTP). Unlike single-host harnesses, the agent derives the target IP from the CIDR scope rather than receiving it directly.

Phase 3: Per-device parallel triage. Phase 3 runs a deterministic scanner pass per device, then spawns one LLM micro-agent per device with only that device’s scan output and a minimal tool set (`http_get`, `cve_search`). Per-device outputs are merged and deduplicated via severity-aware rules. This decomposition keeps each agent’s context constant regardless of network size, at the cost of cross-device correlation, which Phase 5 partially recovers through lateral movement.

Phase 4: Per-vulnerability verification. Phase 4 spawns one micro-agent per Phase 3 finding with full operational tool access (`ssh_login`, `mysql_query`, `mqtt_listen`, `modbus_scan`, `telnet_connect`). Each agent attempts the deepest evidence level achievable (L1–L3, §3) within a bounded turn budget. Phase 3 confirmed findings override Phase 4 errors to avoid erasing directly-observed indicators.

Phase 5: Intrusion and lateral movement. Phase 5 turns the per-device vulnerability inventory into a network-scoped infiltration campaign. The intrusion agent attempts credential spraying, lateral movement along edges present in the topology graph from Phase 1 (reflecting observed or asserted reachability), and crown-jewel access on back-end services unreachable from the entry point. Every hop is recorded with source, destination, protocol, and credentials used, providing the evidence trail used for network-scoped reporting.

Phase 6: Network report. Phase 6 produces a network-scoped report with device inventory, per-device vulnerabilities, attack surface summary, multi-hop intrusion narrative, and recommendations ordered by severity $\times$ exposure. Cross-device aggregation is deliberately deferred to this phase to avoid bloating earlier agents, and the consolidated output enables a human analyst to prioritize remediation across the full network rather than per-host.

5.2. Cost Envelope

For a network with n devices and v findings per device on average, the harness issues $O(nv)$ LLM invocations:

$$\underbrace{1}_{P1} + \underbrace{1}_{P2} + \underbrace{n}_{P3} + \underbrace{nv}_{P4} + \underbrace{1}_{P5} + \underbrace{1}_{P6}$$

Each invocation is bounded by a fixed turn budget (50 for P1–P2, 10 per device for P3, 10 per finding for P4, 30 for

P5, 20 for P6). Cost scales linearly with network size, not combinatorially.

The three structural choices above (topology prior P1, per-device parallelism P3–P4, and the dedicated intrusion phase P5) define the architecture evaluated in §7.

6. Experimental Setup

Infrastructure. All scenarios are deployed on a single Proxmox VE host as LXC containers on a dedicated Linux bridge with an OpenWrt router as gateway; per-scenario deployment and teardown are fully automated via Ansible.

Model. To isolate architectural contributions from model-specific advantages, all LLM-driven systems run on the same general-purpose model: **MiniMax-M2.7** [37]. Security-fine-tuned models are intentionally excluded; this study addresses the architecture axis, not the model fine-tuning axis.

Baselines. We compare LANCE against two adapted LLM-agent baselines, reimplementing published pipeline architectures on the same scenarios and model. The adapted baselines are run in the informed setting; LANCE is reported in both informed and blind modes:

- **CAI-style adapter:** single-agent tool-calling loop, adapted from the CAI [12] architecture.
- **VulnBot-style adapter:** multi-agent with a Penetration Task Graph, adapted from VulnBot [9].

These systems are designed primarily for single-host or single-target workflows, not for full-network graph reasoning. We therefore run each adapted baseline once per ground-truth target device in a scenario, normalize the per-device outputs into the common finding schema, merge them at scenario level, and score the merged output with the same evaluator as LANCE.

Metrics. We report recall (task completion), precision, F1, and CVSS-weighted score, formally defined in §4.5. Evidence levels and hop-depth metadata are retained in the artifact for diagnostic analysis, but they are not reported as aggregate metrics in this paper.

Variance. Reference runs use fixed artifacts, fixed prompts, and temperature 0 decoding. For LANCE, we additionally run a same-artifact repeat in informed mode; §7.2 reports the observed stability. The adapted baselines are reported from one controlled run per target device.

7. Evaluation

7.1. Overall performance

Table 4 reports per-scenario F1 across all four systems. Informed LANCE reaches 0.935 mean F1 (0.959 recall, 0.914 precision); blind LANCE reaches 0.887 mean F1 without any topology prior. Both modes exceed both adapted baselines on every scenario; the baselines never raise F1 above 0.55. The scalability scenarios confirm the trend: S11 (15 devices) reaches F1 = 0.875 and S12 (35 devices) reaches F1 = 0.942.

TABLE 4. PER-SCENARIO F1 ON ALL 12 SCENARIOS (REFERENCE RUN).

Scenario	Informed	Blind	CAI	VulnBot
S1	0.923	0.880	0.153	0.153
S2	0.923	0.923	0.353	0.445
S3	0.947	0.941	0.000	0.000
S4	0.944	0.909	0.363	0.435
S5	1.000	0.929	0.500	0.500
S6	1.000	1.000	0.400	0.400
S7	0.965	0.889	0.500	0.445
S8	0.933	0.923	0.546	0.353
S9	0.952	0.857	0.167	0.286
S10	0.815	0.769	0.143	0.143
S11	0.875	0.857	0.414	0.414
S12	0.942	0.762	0.242	0.307
Mean	0.935	0.887	0.315	0.323

$F1 = 0$ means that the normalized output contained no finding matching the scenario ground truth under the common evaluator.

7.2. Rerun consistency (informed mode)

Informed LANCE was executed twice on the same artifact set to measure output stability (Table 5). The mean delta is -0.017 F1: S7 shows the largest drop (-0.182) and S10 the largest gain ($+0.145$), while seven scenarios change by less than 0.05. Blind LANCE was executed once per scenario.

TABLE 5. INFORMED RERUN DELTA ($\Delta = \text{RUN B} - \text{RUN A}$).

Scenario	Run A	Run B	Δ
S1	0.923	0.960	+0.037
S2	0.923	0.833	-0.090
S3	0.947	0.971	+0.024
S4	0.944	0.788	-0.156
S5	1.000	0.965	-0.035
S6	1.000	0.970	-0.030
S7	0.965	0.783	-0.182
S8	0.933	0.963	+0.030
S9	0.952	0.952	± 0.000
S10	0.815	0.960	+0.145
S11	0.875	0.957	+0.082
S12	0.942	0.919	-0.023
Mean	0.935	0.918	-0.017

7.3. Per-scenario metric detail

Table 6 reports recall, precision, F1, and CVSS-weighted score per scenario for both LANCE modes. Informed recall is perfect on six scenarios (S1, S3, S5, S6, S7, S8); precision drops on S10 (sparse coverage of a large flat topology). Blind mode trades recall for higher precision on gateway-centric scenarios (S3, S4, S8), confirming that active discovery raises precision at the cost of recall. The CVSS-weighted score is systematically lower in blind mode (73.8% vs. 86.4%), reflecting greater severity estimation error when no topology prior is available.

TABLE 6. PER-SCENARIO RECALL, PRECISION, F1, AND CVSS SCORE FOR BOTH LANCE MODES.

Scenario	Informed				Blind			
	Rec	Prec	F1	CVSS%	Rec	Prec	F1	CVSS%
S1	1.000	0.857	0.923	100.0	0.917	0.846	0.880	77.4
S2	0.923	0.923	0.923	85.5	0.923	0.923	0.923	78.2
S3	1.000	0.900	0.947	90.2	0.889	1.000	0.941	81.0
S4	0.944	0.944	0.944	80.4	0.833	1.000	0.909	68.9
S5	1.000	1.000	1.000	88.6	0.867	1.000	0.929	74.3
S6	1.000	1.000	1.000	88.0	1.000	1.000	1.000	85.5
S7	1.000	0.933	0.965	85.5	0.857	0.923	0.889	72.5
S8	1.000	0.875	0.933	85.7	0.857	1.000	0.923	73.3
S9	0.909	1.000	0.952	85.9	0.818	0.900	0.857	77.8
S10	0.846	0.786	0.815	71.3	0.769	0.769	0.769	64.6
S11	0.913	0.840	0.875	84.8	0.783	0.947	0.857	67.4
S12	0.976	0.911	0.942	90.6	0.762	0.762	0.762	65.0
Mean	0.959	0.914	0.935	86.4	0.856	0.922	0.887	73.8

7.4. Cross-benchmark comparison on single-host corpora

To probe transfer beyond IoT multi-hop scenarios, we run LANCE and the adapted baselines on two established single-host benchmarks. Cross-corpus comparison is indicative rather than definitive: each paper uses a different model, oracle, and task definition.

AutoPenBench. AutoPenBench defines 33 flag-based tasks with published E2E rates. We run *blind* mode (endpoint and service metadata only) and *informed* mode (also the case identifier and vulnerability label). Table 7 separates published paper results from our controlled MiniMax-M2.7 runs.

TABLE 7. AUTOPENBENCH TRANSFER RESULTS (33 TASKS).

System	Model	Sub-task	Flags (E2E)
<i>Published results (%; denominator/subsets differ)</i>			
Llama baseline [9], [14]	Llama 3.1-405B	49.1%	9.1%
GPT-4o baseline [9], [14]	GPT-4o	n/r	21.2%
VulnBot [9]	Llama 3.1-405B	69.1%	30.3%
xOffense [14]	Qwen3-32B LoRA	79.2% [¶]	72.7%
<i>This work (all 33 tasks, MiniMax-M2.7; %, count/33)</i>			
CAI adapter [12]	MiniMax-M2.7	12.1% (4/33) [†]	6.1% (2/33)
VulnBot adapter [9]	MiniMax-M2.7	18.2% (6/33) [†]	15.2% (5/33)
LANCE informed	MiniMax-M2.7	51.5% (17/33)[†]	27.3% (9/33)
LANCE blind	MiniMax-M2.7	39.4% (13/33) [†]	21.2% (7/33)

[†] Sub-task = useful findings (actionable exploit evidence, no flag required). Published rows report percentages over their own (possibly undisclosed) subsets and are not necessarily comparable to the 33-task denominator used here. [¶] xOffense sub-task is best-of-five. n/r = not reported. **Bold** = best result in this work.

Under the same model and oracle, LANCE leads on both metrics: informed captures 9 flags and 17 useful findings, blind captures 7 and 13, versus 5 flags and 6 useful findings for VulnBot.

Vulhub. Vulhub has no uniform flag oracle, so we report useful-finding lower bounds. Table 8 shows our controlled 328-case snapshot alongside published results for context.

TABLE 8. VULHUB TRANSFER RESULTS. OUR RUNS COVER ALL 328 CASES.

System	Model	Cases	Result
<i>Published (% oracle and subset differ)</i>			
PentestAgent [11]	n/r	67	74.2%
CheckMate [15]	Sonnet 4.5	120	88%
<i>This work, useful-finding oracle (/328)</i>			
LANCE informed	MiniMax-M2.7	328	99/328 (30.2%)
LANCE blind	MiniMax-M2.7	328	92/328 (28.0%)
CAI adapter [12]	MiniMax-M2.7	328	170/328 (51.8%)
VulnBot adapter [9]	MiniMax-M2.7	328	92/328 (28.0%)

*Published rows use undisclosed subsets and are not comparable to the 328-case denominator used here. Useful findings = confirmed exploits or probable vulnerabilities with machine-readable evidence. **Bold** = best result overall.*

On Vulhub, CAI leads with 170/328 useful findings; LANCE informed reaches 99/328, blind reaches 92/328 (equal to VulnBot). CAI’s advantage reflects its single-host exploit focus, which is orthogonal to the multi-device topology reasoning evaluated in IoTChainBench.

8. Discussion and Limitations

Why the gap holds on IoTChainBench. LANCE outperforms both adapted baselines on every IoTChainBench scenario because the baselines treat the network as a flat list of independent targets. The clearest evidence is S3: a gateway-centric topology where back-end services are unreachable without a prior traversal step, causing both baselines to produce no true-positive matches ($F1 = 0$) while LANCE exceeds $F1 = 0.94$. The controlled model and evaluator rule out model-specific advantages; the gap is architectural. Note that this advantage is specific to multi-hop network scenarios: on single-host corpora (AutoPenBench, Vulhub), results are more nuanced — see below.

Transfer results in context. On AutoPenBench and Vulhub, LANCE is competitive with adapted single-host agents under the same model. CAI leads on the Vulhub useful-finding oracle (51.8% vs 30.2%), which reflects its stronger single-host exploit focus. This is expected and does not contradict the main result: neither AutoPenBench nor Vulhub contains pivot-dependent vulnerabilities, so the topology prior and lateral-movement phases that drive the IoTChainBench gain are simply not exercised.

Limitations. (i) *Simulated infrastructure.* Scenarios run on Linux containers; real constrained hardware (bootloaders, RTOS, firmware) introduces attack surfaces not covered here. (ii) *Run-to-run variance.* Same-artifact reruns confirm that informed mode is stable ($\Delta F1 = -0.017$ mean, Table 5). Blind mode was executed once per scenario; full confidence intervals over repeated API calls remain future work. (iii) *Evaluator leniency.* Bonus categories are accepted as non-FP without ground-truth matches, which may inflate precision on scenarios with broad bonus lists. (iv) *Prompt sensitivity.* A single prompt family is used across all models;

per-model tuning may change absolute scores. (v) *Static topologies.* The five architectural patterns are fixed; real deployments include device families not represented here and evolve over time. (vi) *Ground-truth injection scope.* Ground truth covers vulnerabilities tractably injectable via Ansible; physical-layer and supply-chain weaknesses are not represented, potentially underestimating the difficulty of real IoT audits.

9. Ethical Considerations

All experiments run on an isolated Proxmox cluster with no route to external networks. Benchmark agents do not probe live hosts, production services, or third-party infrastructure.

All vulnerabilities are deterministically injected into our own lab containers from public vulnerability classes and documented misconfiguration patterns. The work does not introduce or disclose new third-party vulnerabilities; no vendor notification process is triggered. External corpora are run locally from their public artifacts and used only for measurement.

The released artifact contains deployment playbooks, ground truth, evaluator code, and run logs. It includes no private targets, credentials, or novel exploit payloads, and is intended for security research and benchmark reproduction in controlled lab environments.

10. Conclusion

Single-host benchmarks cannot capture the pivot chains, protocol roles, and reachability dependencies that define real IoT risk. We introduced IoTChainBench, a 12-scenario network-scale benchmark with deterministic vulnerability injection and per-vulnerability ground truth, and LANCE, a six-phase network-native harness that decomposes pentesting at device and vulnerability granularity.

Under a controlled comparison with the same model and evaluator, LANCE achieves 0.935 mean $F1$ and 86.4% CVSS-weighted score, outperforming every adapted baseline on every scenario. The sharpest evidence is S3: both baselines produce no true-positive matches ($F1 = 0$) on a gateway-centric topology that requires a single pivot, while LANCE exceeds $F1 = 0.94$. The gain scales: $F1 = 0.942$ on a 35-device scenario confirms that bounded per-device context does not degrade with network size.

These results show that network-aware context is a necessary condition for IoT pentesting, not a refinement. We release all scenarios, playbooks, ground truth, and evaluation code to support reproducible research in this setting. All artifacts are available at ANONYMIZED_URL.

References

- [1] M. Antonakakis *et al.*, “Understanding the Mirai botnet,” in *Proc. 26th USENIX Security Symp.*, 2017.
- [2] W. Turton, “Hackers breach thousands of security cameras, exposing Tesla, jails, hospitals,” *Bloomberg*, Mar. 2021.

- [3] R. Langner, “Stuxnet: Dissecting a cyberwarfare weapon,” *IEEE Security & Privacy*, vol. 9, no. 3, pp. 49–51, 2011.
- [4] A. Happe and J. Cito, “Getting pwn’d by AI: Penetration testing with large language models,” in *Proc. ACM ESEC/FSE*, 2023.
- [5] G. Deng *et al.*, “PentestGPT: Evaluating and harnessing large language models for automated penetration testing,” in *Proc. 33rd USENIX Security Symp.*, 2024.
- [6] A. Happe *et al.*, “HackingBuddyGPT: An LLM-driven pentest agent,” 2024. [Online]. Available: <https://github.com/ipa-lab/hackingBuddyGPT>
- [7] J. Xu *et al.*, “AutoAttacker: A large language model guided system to implement automatic cyber-attacks,” *arXiv:2403.01038*, 2024.
- [8] O. V. Aguinaga, “Nebula: An LLM-powered penetration testing assistant,” *arXiv:2412.00006*, 2024.
- [9] H. He *et al.*, “VulnBot: Autonomous penetration testing for a multi-agent collaborative framework,” *arXiv:2501.13411*, 2025.
- [10] Y. Ginige *et al.*, “AutoPentester: An LLM agent-based framework for automated pentesting,” *arXiv:2510.05605*, 2025.
- [11] X. Shen *et al.*, “PentestAgent: Incorporating LLM agents to automated penetration testing,” in *Proc. 20th ACM Asia Conf. Computer and Communications Security (ASIA CCS)*, 2025. doi: 10.1145/3708821.3733882.
- [12] V. Mayoral-Vilches *et al.*, “CAI: An open, bug bounty-ready cybersecurity AI,” *arXiv:2504.06017*, 2025.
- [13] Anonymous, “Pentest-R1: Towards autonomous penetration testing reasoning optimized via two-stage reinforcement learning,” *arXiv:2508.07382*, 2025.
- [14] P. D. Luong *et al.*, “xOffense: An autonomous multi-agent framework for penetration testing with domain-adapted large language models,” *arXiv:2509.13021*, 2025.
- [15] L. Wang, X. Shi, Z. Li, Y. Jiang, S. Tan, Y. Jiang, J. Cheng, W. Chen, X. Shen, Z. Li, and Y. Chen, “Automated penetration testing with LLM agents and classical planning,” *arXiv:2512.11143*, 2025.
- [16] L. Gioacchini, M. Mellia, I. Drago, A. Delsanto, G. Siracusanu, and R. Bifulco, “AutoPenBench: Benchmarking generative agents for penetration testing,” *arXiv:2410.03225*, 2024.
- [17] Vulnhub contributors, “Vulnhub: Pre-built vulnerable Docker environments for learning and reproducing CVEs,” [Online]. Available: <https://github.com/vulnhub/vulnhub>
- [18] V. Mayoral-Vilches *et al.*, “CAIBench: A meta-benchmark for evaluating cybersecurity AI agents,” *arXiv:2510.24317*, 2025.
- [19] M. Shao *et al.*, “NYU CTF Bench: A scalable open-source benchmark dataset for evaluating LLMs in offensive security,” in *Proc. NeurIPS Datasets and Benchmarks Track*, 2024.
- [20] A. K. Zhang *et al.*, “Cybench: A framework for evaluating cybersecurity capabilities and risk of language models,” *arXiv:2408.08926*, 2024.
- [21] Ludus Cloud, “Ludus: Cyber range automation on Proxmox,” [Online]. Available: <https://ludus.cloud>
- [22] M3, “GOAD: Game of Active Directory,” [Online]. Available: <https://github.com/Orange-Cyberdefense/GOAD>
- [23] C. Phillips and L. P. Swiler, “A graph-based system for network-vulnerability analysis,” in *Proc. New Security Paradigms Workshop*, 1998.
- [24] O. Sheyner, J. Haines, S. Jha, R. Lippmann, and J. M. Wing, “Automated generation and analysis of attack graphs,” in *Proc. IEEE Symp. Security and Privacy*, 2002.
- [25] X. Ou, S. Govindavajhala, and A. W. Appel, “MulVAL: A logic-based network security analyzer,” in *Proc. USENIX Security Symp.*, 2005.
- [26] OWASP Foundation, “OWASP Internet of Things Top 10,” 2024. [Online]. Available: <https://owasp.org/www-project-internet-of-things-top-10/>
- [27] MITRE Corporation, “MITRE ATT&CK for ICS,” 2024. [Online]. Available: <https://attack.mitre.org/matrices/ics/>
- [28] K. Stouffer *et al.*, “Guide to operational technology security,” National Institute of Standards and Technology, NIST SP 800-82 Rev. 3, 2023.
- [29] S. Garcia, A. Parmisano, and M. J. Erquiaga, “IoT-23: A labeled dataset with malicious and benign IoT network traffic,” Stratosphere Laboratory, 2020. [Online]. Available: <https://www.stratosphereips.org/datasets-iot23>
- [30] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood, and A. Anwar, “TON_IoT telemetry dataset: A new generation dataset of IoT and IIoT for data-driven intrusion detection systems,” *IEEE Access*, vol. 8, pp. 165130–165150, 2020.
- [31] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke, “Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning,” *IEEE Access*, vol. 10, pp. 40281–40306, 2022.
- [32] A. Costin, J. Zaddach, A. Francillon, and D. Balzarotti, “A large-scale analysis of the security of embedded firmwares,” in *Proc. USENIX Security Symp.*, 2014.
- [33] D. D. Chen, M. Egele, M. Woo, and D. Brumley, “Towards automated dynamic analysis for Linux-based embedded firmware,” in *Proc. NDSS*, 2016.
- [34] M. Kim, D. Kim, E. Kim, S. Kim, Y. Jang, and Y. Kim, “FirmAE: Towards large-scale emulation of IoT firmware for dynamic analysis,” in *Proc. Annual Computer Security Applications Conf. (ACSAC)*, 2020.
- [35] OWASP, “IoTGoat: A deliberately insecure IoT firmware,” 2024. [Online]. Available: <https://github.com/OWASP/IoTGoat>
- [36] T. Abramovich *et al.*, “EnIGMA: Interactive tools substantially assist LM agents in solving CTF challenges,” in *Proc. ICML*, 2025.
- [37] MiniMax, “MiniMax-M2.7: A general-purpose large language model,” 2026. [Online]. Available: <https://www.minimax.io/models/text/m27>

LLM Usage Statement

LLMs were used for editorial assistance while preparing this manuscript, and all generated text was inspected by the authors to ensure accuracy, originality, and consistency with the underlying artifacts. LLMs are also part of the studied methodology: LANCE uses tool-calling LLM agents for network reconnaissance, vulnerability analysis, exploit verification, intrusion attempts, and report generation, as described in §5, §6, and §7.